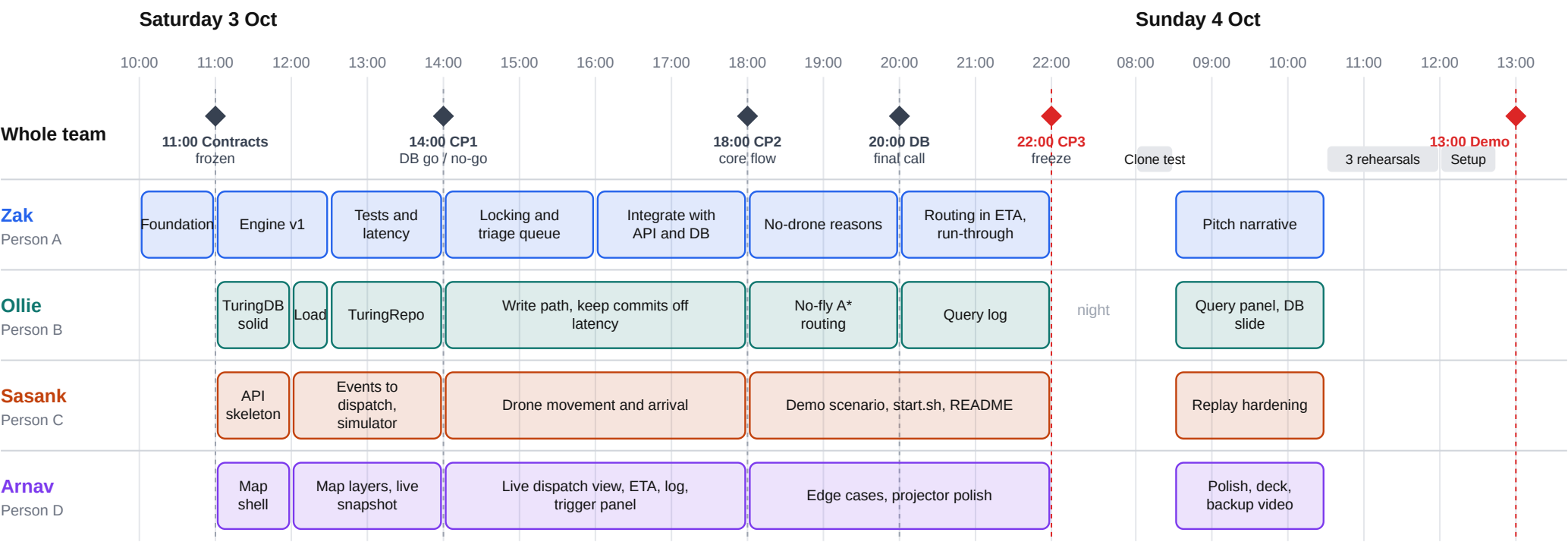


EDTH Weekend Plan

Battlefield medical resupply and drone dispatch on TuringDB · Saturday 3 to Sunday 4 October 2026 · demo at 13:00 Sunday

- Zak (A): Dispatch engine and contracts
- Ollie (B): TuringDB and routing
- Sasank (C): Backend, simulator and live updates
- Arnav (D): Map dashboard and pitch deck

The goal: by 22:00 tonight the full demo runs end to end. Sunday is only polish, rehearsal and the pitch. Each person owns one layer, and the four layers meet only at the contracts frozen at 11:00.



WHEN	WHAT MUST BE TRUE	WHEN	WHAT MUST BE TRUE
Sat 11:00	Contracts frozen in the repo: schema, GraphRepo interface, event and WebSocket formats	Sat 22:00	Demo scenario runs twice in a row on one laptop; feature freeze
Sat 14:00	Checkpoint 1: dispatch through the API in under 1 s. TuringDB go or no-go	Sun 08:00	Fresh clone on a clean laptop runs <code>./start.sh</code> and the demo
Sat 18:00	Checkpoint 2: trigger to drone flight to arrival, live ETA and latency counter, on <code>main</code>	Sun 10:30	Feature freeze, tag <code>demo-v1</code> ; only demo-blocking fixes after this
Sat 20:00	TuringDB final call: if it can't run the demo, demo on the in-memory repo	Sun 13:00	Demo

Times are CEST. Zak starts at 10:00; teammates are assumed to join by 11:00. Dashed lines mark checkpoints (CP), red ones the two hard deadlines.



Zak owns the decision at the heart of the demo: given an event, which drone goes and when it arrives, in well under a second.

10:00–11:00 Foundation

- ☐ Everything in How we start (page 6): skeleton, venv, TuringDB smoke test, contracts, seed data, push

Done when teammates can pull `main` and import `models`, `repo` and `seed`

11:00–12:30 Dispatch engine v1 (`dispatch.py`)

- ☐ Needs table: CASUALTY + CRITICAL needs 1 tourniquet, 2 blood_oneg, 1 hemostatic_gauze; CASUALTY + WOUNDED needs 1 tourniquet, 1 chest_seal; LOW_STOCK needs the items named in the event
- ☐ `InMemoryRepo` implementing every `GraphRepo` method from seed data
- ☐ Candidate filter: drone IDLE, carries every needed item in enough quantity
- ☐ Range check: distance to recipient plus distance from recipient to nearest depot, times 1.2 safety margin, must fit in remaining range
- ☐ ETA: haversine distance divided by drone speed; pick the lowest ETA

Done when `dispatch(event)` returns a `Dispatch` or a `NoDispatch` with a reason

12:30–14:00 Tests and latency

- ☐ pytest: picks the nearest suitable drone; skips a drone with too little stock; skips one out of range; no drone gives a reason
- ☐ pytest: two events dispatched at the same moment get two different drones
- ☐ Time `dispatch()` with `time.perf_counter()` and log it; target under 50 ms in memory

Done when tests green on `main`, ready for checkpoint 1

14:00–16:00 Concurrency and triage queue

- ☐ `claim_drone` guarded by one `asyncio.Lock`, so the check and the claim can't interleave
- ☐ Priority queue for unserved requests: CRITICAL first, then WOUNDED, then LOW_STOCK, oldest first within a level
- ☐ When a drone becomes IDLE, drain the queue before anything else

Done when firing three CRITICAL events with two free drones dispatches two and queues one

16:00–18:00 Integrate

- ☐ Pair with Sasank to call the engine from `POST /events`
- ☐ Pair with Ollie to swap in `TuringRepo`; keep the dispatch path to one candidate query and do graph writes after the dispatch message is sent

Done when checkpoint 2 passes on one laptop

18:00–20:00 No-drone handling

- ☐ Reasons: no drone carries the item, all suitable drones busy (queued, with position), suitable drones out of range
- ☐ `nearest_alternative`: the closest drone that could serve after resupplying at a depot, with its longer ETA

Done when each reason shows up on Arnav's map

20:00–22:00 Routing and run-through

- ☐ Use Ollie's route length instead of straight-line distance for ETA and range
 - ☐ Run the demo scenario end to end twice, fix what breaks
 - ☐ Draft the pitch outline
-

Sunday

Sunday

- 08:30–10:30 ☐ Pitch narrative and delivery; lead the three rehearsals
-

Before 11:00 Zak works alone on the foundation. Checkpoints at 14:00, 18:00 and 22:00 are for the whole team: stop, pull main, run everything on one laptop.



Ollie makes TuringDB the real store behind GraphRepo by 14:00 and builds the no-fly routing in the evening; nobody else waits on either, because the in-memory repo covers until then.

11:00–12:00 Get TuringDB solid

- ☐ Read Zak's smoke-test numbers, then reproduce: `pip install turingdb, turingdb -demon`, connect with `TuringDB(host="http://localhost:6666")`
- ☐ Confirm the write cycle: `new_change()`, `checkout(change=...)`, `CREATE` or `SET`, `COMMIT`, `CHANGE SUBMIT`, `checkout()` back to main
- ☐ Measure: one `MATCH` read, one single-property `SET` commit; post both in ms to the team
- ☐ Check the embedded mode `TuringDB(type="embedded", data_dir=...)` in case it avoids a network hop

Done when a written-down list of what works, what doesn't, and the timings

12:00–12:30 Load both graphs

- ☐ `scripts/load_turingdb.py` creates graphs `personnel` and `logistics` and loads `seed.py`
- ☐ Personnel: `Soldier`, `Medic`, `Unit` nodes; `MEMBER_OF`, `ATTACHED_TO` edges; medic stock as properties such as `stock_blood_oneg`
- ☐ Logistics: `Drone`, `Depot`, `SupplyItem` nodes; `BASED_AT` and `CARRIES {qty}` edges

Done when the TuringDB UI (`turingdb -ui`, port 8080) shows both graphs

12:30–14:00 TuringRepo

- ☐ `find_candidate_drones` as one `MATCH (d:Drone)-[c:CARRIES]->(s:SupplyItem) WHERE d.status = 'IDLE' RETURN ...`, filtered for quantities in Python
- ☐ Run Zak's repo tests against `TuringRepo` as well as `InMemoryRepo`

Done when tests pass, for the 14:00 go or no-go

14:00–18:00 Write path

- ☐ `update_drone`, `update_person`, `create_dispatch` (a `DISPATCHED_TO` edge with eta and timestamp), `complete_dispatch`
- ☐ If a commit takes over about 100 ms, move writes to a background writer task so they never sit on the dispatch path
- ☐ Drone claiming stays in Zak's in-process lock; TuringDB records the result

Done when after a demo run, the graph shows the dispatches and updated stock

18:00–20:00 No-fly routing (`routing.py`)

- ☐ Waypoint grid at about 500 m spacing over the area, dropping points inside no-fly polygons
- ☐ Store waypoints and `ROUTE_SEGMENT` edges in the logistics graph; load once at startup
- ☐ A* in Python: `route(a, b)` returns the list of `[lat, lon]` points and the length in metres

Done when a route between two points on either side of a no-fly zone bends around it

- ☐ 20:00: TuringDB final call with Zak
-

20:00–22:00 Query log

- ☐ Record each Cypher query with its duration, and send it over the WebSocket as a new `query_log` message (agree the shape with Sasank and Arnav)
-

Sunday Sunday

- ☐ 08:30–10:30 "What the graph query did" panel content and the TuringDB slide
-

Times assume you join by 11:00; if you arrive later, start at your first block and shift the rest. Checkpoints at 14:00, 18:00 and 22:00 are for the whole team: stop, pull main, run everything on one laptop.



Sasank owns everything that moves: events in, messages out over the WebSocket, drones flying along their routes, and the scripted demo.

11:00–12:00 API skeleton (`main.py`)

- ☐ FastAPI app with `GET /health`, `GET /state`, `POST /events`, and `/ws` with a small connection manager that broadcasts to every client
- ☐ `/ws` sends `snapshot` on connect, built from the repo

Done when `uvicorn backend.app.main:app --reload` runs and a test client receives `snapshot`

12:00–14:00 Events to dispatch

- ☐ `POST /events`: stamp `received_ts`, write the person update via the repo, call `dispatch()`, broadcast event then `dispatch` or `no_dispatch`
- ☐ `latency_ms` = time from request received to dispatch decided, measured with `time.perf_counter()`; log every one
- ☐ Use a stub `dispatch()` that returns the first idle drone until Zak's engine lands
- ☐ `sim/simulator.py` posts events over HTTP, so real telemetry can replace it later; random mode plus `--scenario demo`

Done when the simulator fires a CRITICAL casualty and a WebSocket client sees `dispatch`

14:00–18:00 Drone movement and arrival

- ☐ Background async task ticking twice a second, with a time multiplier (say 10x) so demo flights last under a minute
- ☐ Move each `EN_ROUTE` drone along its route, reduce battery, broadcast `drone_update` with the remaining `eta_s`
- ☐ On arrival: broadcast `delivered`, update the recipient's stock via the repo, set the drone `RETURNING` to the nearest depot, then `IDLE`, then ask the engine to drain its queue
- ☐ Personnel drift slowly in the simulator so the map looks alive

Done when checkpoint 2, a full flight from trigger to drone back at base

18:00–22:00 Demo scenario and start script

- ☐ Scripted scenario with a fixed seed: squad on patrol; at +5 s a soldier goes CRITICAL; at +5.2 s a nearby medic reports low blood; at +6 s a second CRITICAL in another squad; one route crosses a no-fly zone
- ☐ `POST /scenario/{name}` starts it from the dashboard
- ☐ `start.sh`: start TuringDB, load seed data, start uvicorn serving the frontend, print the URL
- ☐ README: install and run in under five commands

Done when a fresh clone runs the demo with `./start.sh`

Sunday Sunday

- 08:30–10:30 ☐ Harden the replay so the scenario runs identically every time; test on the demo laptop
-

Times assume you join by 11:00; if you arrive later, start at your first block and shift the rest. Checkpoints at 14:00, 18:00 and 22:00 are for the whole team: stop, pull main, run everything on one laptop.



Arnav owns what the judges see: one map page that tells the whole story at a glance, readable on a projector.

11:00–12:00 Map shell

- ☐ `frontend/index.html` with Leaflet from a CDN and plain JavaScript, no build step
- ☐ Map centred on Grafenwöhr training area (about 49.70 N, 11.93 E), zoom 13
- ☐ `frontend/mock/snapshot.json` matching the `snapshot` message in the contracts

Done when the page opens from disk and shows the mock data

12:00–14:00 Layers

- ☐ Soldiers coloured by status: OK green, WOUNDED amber, CRITICAL red
- ☐ Medics with a distinct icon and a popup listing stock, low items highlighted
- ☐ Drones by status, depots, no-fly zones as red polygons
- ☐ Connect to `/ws` and render `snapshot`; fall back to the mock file if the socket is down

Done when checkpoint 1 shows the live snapshot from Sasank's API

14:00–18:00 Live dispatch view

- ☐ On `dispatch`: draw the route, show an ETA badge on the drone that counts down every second and resyncs on each `drone_update`
- ☐ Move drones on `drone_update`; on `delivered`, flash the recipient and clear the route
- ☐ Event log panel, newest first, with event-to-dispatch ms on each line
- ☐ Latency counter in the header: last dispatch and rolling average in ms
- ☐ Trigger panel: pick a soldier and set CRITICAL, or pick a medic and an item to report low stock; both `POST /events`

Done when checkpoint 2, a trigger click ends with a drone flying and its ETA counting down

18:00–22:00 Edge cases and projector polish

- ☐ `no_dispatch`: a clear banner with the reason and the suggested alternative
 - ☐ `queue`: a small panel listing waiting requests in triage order
 - ☐ Rerouted route visibly bending around the no-fly zone
 - ☐ "Run demo scenario" button calling `POST /scenario/demo`
 - ☐ Dark theme, large fonts, test at 1920 by 1080
-

Sunday Sunday

- 08:30–10:30** ☐ Visual polish, pitch deck with Zak, and a backup screen recording of the best rehearsal
-

Times assume you join by 11:00; if you arrive later, start at your first block and shift the rest. Checkpoints at 14:00, 18:00 and 22:00 are for the whole team: stop, pull main, run everything on one laptop.

How we start

Zak lays the shared foundation in the first hour, so each teammate can start their own work within ten minutes of arriving.

- 1. **Repo skeleton (10 min).** Create the layout below on `main` and push it.
- 2. **Environment (5 min).** `python3 -m venv .venv && source .venv/bin/activate` then `pip install fastapi "uvicorn[standard]" turingdb pytest httpx`
- 3. **TuringDB smoke test (15 min).** The biggest risk, so test it first. Run `turingdb -demon`, then create a graph, open a change, `CREATE` a node, `COMMIT`, `CHANGE SUBMIT`, `MATCH` it back and `SET` a property. Time each step and share the numbers with Ollie.
- 4. **Contracts (20 min).** `models.py`, the `GraphRepo` protocol in `repo.py`, and `contracts/messages.md`.
- 5. **Seed data (10 min).** 3 squads (20 personnel, 3 medics), 8 drones, 3 depots and 2 no-fly zones around Grafenwöhr training area (about 49.70 N, 11.93 E).
- 6. **Push and brief.** Each teammate gets a five-minute walkthrough of the contracts, then starts the first block on their page.

backend/app/main.py	FastAPI app and /ws	Sasank
backend/app/models.py	Person, Drone, Depot, Event, Dispatch	Zak
backend/app/repo.py	GraphRepo protocol + InMemoryRepo	Zak
backend/app/turing_repo.py	TuringDB implementation	Ollie
backend/app/dispatch.py	dispatch engine	Zak
backend/app/routing.py	waypoint graph + A*	Ollie
backend/app/seed.py	seed data shared by both repos	Zak
sim/simulator.py	event simulator, posts to /events	Sasank
frontend/index.html	Leaflet dashboard, no build step	Arnav
contracts/ tests/ start.sh		

TuringDB facts from its docs: server on port 6666; every write goes through a change that is committed and submitted; `SET` updates properties; every node needs a label. So the engine claims drones with an in-process lock and writes to the graph just after the dispatch goes out, keeping commits off the latency path.

Contracts, frozen at 11:00

Graph schema (Ollie)

Two graphs, `personnel` and `logistics`. Ids carry a type prefix: `sol-07`, `med-02`, `drn-04`, `dep-01`. Items: `tourniquet`, `blood_oneg`, `chest_seal`, `hemostatic_gauze`, `morphine_autoinjector`.

GraphRepo (Zak and Ollie)

- `get_person`, `update_person`, `list_personnel`
- `find_candidate_drones(items)`: idle drones with enough of every item, in one query
- `claim_drone(drone_id, request_id) -> bool`, atomic
- `create_dispatch`, `complete_dispatch`, `update_drone`, `list_drones`, `list_depots`, `list_no_fly_zones`

Event in (Sasank): POST /events

```
{"event_id": "evt-123", "type": "CASUALTY",
"subject_id": "sol-07", "severity": "CRITICAL",
"items": {"blood_oneg": 2},
"lat": 49.70, "lon": 11.93, "ts": 1696320000.123}
```

WebSocket out (Sasank to Arnav): /ws

TYPE	SENT WHEN	KEY FIELDS
snapshot	on connect	personnel, drones, depots, no-fly zones
event	event received	event + received_ts
dispatch	drone assigned	request_id, drone_id, recipient_id, route, eta_s, latency_ms
no_dispatch	no drone fits	request_id, reason, nearest_alternative
drone_update	each tick, 2 per s	drone_id, lat, lon, status, eta_s
delivered	drone arrives	request_id, drone_id, recipient_id, items
queue	queue changes	pending requests in triage order

Also `GET /state` (same as snapshot) and `POST /scenario/{name}`. Changing a contract after 11:00 needs a two-minute huddle with whoever uses it.

Sunday until the demo

TIME	EVERYONE
08:00–08:30	Fresh clone on a clean laptop, run the start script and the demo scenario; whatever breaks is the first fix
08:30–10:30	Bug fixes and polish only. Zak and Arnav draft the pitch, Sasank hardens the replay, Ollie builds the graph query panel and TuringDB slide. Merge only after a second person checks
10:30	Feature freeze, tag <code>demo-v1</code>
10:30–12:00	Three full timed rehearsals: 3-minute pitch plus live demo, one person plays judge. Arnav records a backup video of the best run
12:00–12:45	Final setup on the demo laptop, offline check, backup video ready, notifications muted
13:00	Demo

Risks and the TuringDB fallback

- **Stuck 45 minutes** on any single TuringDB issue: Ollie tells Zak, everyone else keeps building on the in-memory repo.
- **14:00 go or no-go:** TuringRepo must pass the same repo tests as the in-memory one (seed, candidate query, property update, claim). If not, the integration runs on in-memory while Ollie keeps going.
- **20:00 final call:** if TuringDB can't run the demo, we demo on in-memory and pitch TuringDB through the schema and query panel. Nobody touches the database layer after 20:00.
- **Latency over a second:** measured from the first hour; one candidate query per dispatch, ETA maths in Python, graph writes after the dispatch message.
- **Integration drift:** contracts frozen at 11:00; checkpoints at 14:00, 18:00 and 22:00 run everything on one laptop.
- **Demo flakiness:** the scripted scenario uses a fixed random seed.
- **Working rule:** merge small working increments to `main` at least every 90 minutes.

Cut list if we fall behind

Cut from the top first, decided at a checkpoint:

1. Multi-drone dispatch and relaying
2. Triage preemption (keep the CRITICAL-first queue)
3. Replay mode
4. Dynamic rerouting mid-flight (keep the static no-fly route at dispatch time)
5. Live graph query panel (replace with a slide)
6. Vitals and unit membership on the map

Never cut: event to dispatch to live ETA to arrival, the latency counter, two simultaneous emergencies taking different drones, and the no-drone reason.

The demo we are building toward

A squad is patrolling. One soldier is hit and goes CRITICAL while a nearby medic reports low blood. The dashboard shows both events, dispatches the closest suitable drones in under a second, ETAs count down, one drone routes around a no-fly zone, and a second simultaneous emergency takes a different drone. The latency counter is visible throughout.